DEPLOY PROCEDURE

RelayMart

From clone to Compose, two servers, ECR, ECS, and EKS

Follow the parts in order. Part A proves the application on one server. Part B splits the frontend onto a second server. Parts C, D, and E move the same images to AWS. Do not open ECS until the Compose health check and a register call have succeeded.

1. What gets deployed

The browser loads the shop from the frontend. The page then calls only the API gateway. The gateway calls the backend services. Services do not call each other, except by publishing events to RabbitMQ.

Piece	Where it runs	Port
Frontend shop	its own server, or with the stack	4173
API gateway	backend server	4000
15 backend services	backend server	4001-4015
Postgres, Redis, RabbitMQ	backend server	not public
Kibana	backend server	5601
Grafana	backend server	3000

Images, one each: frontend, gateway, catalog, search, pricing, inventory, cart, order, payment, shipping, review, user, notification, recommendation, auth, verification, tracking.

You do not edit service source to deploy. The only values you set are URLs, the RDS password, and AWS identifiers.

2. Install on the server

Size

- One server for everything: 4 GB RAM minimum, 8 GB if Kibana must stay up. 30 GB disk.
- Two servers: backend 4 GB or more. Frontend 1 GB is enough.
- Ubuntu 22.04 or Amazon Linux 2023. Open SSH from your IP only.

Ubuntu

```
sudo apt-get update
sudo apt-get install -y docker.io docker-compose-v2 git
sudo usermod -aG docker $USER
```

Amazon Linux 2023

```
sudo dnf update -y
sudo dnf install -y docker git
sudo systemctl enable --now docker
sudo usermod -aG docker ec2-user
sudo mkdir -p /usr/libexec/docker/cli-plugins
VER=v2.32.4
sudo curl -fsSL -o /tmp/docker-compose \
  https://github.com/docker/compose/releases/download/${VER}/docker-compose-linux-x86_
64
sudo mv /tmp/docker-compose /usr/libexec/docker/cli-plugins/docker-compose
```

```
sudo chmod +x /usr/libexec/docker/cli-plugins/docker-compose
```

Log out and SSH back in. Then check:

```
docker version
docker compose version
git --version
```

Expected: Both docker and docker compose print a version. If compose is missing, the rest of this guide will fail.

Node, Terraform, and kubectl are not required for Parts A and B. Install Terraform 1.6+ before Part D. Install kubectl and the AWS CLI before Part E. The AWS CLI is also required for Part C.

3. Ports

Server	Port	Who can connect
Frontend	4173	your users
Backend	4000	your users, because the browser calls it
Backend	5601	you, for Kibana
Backend	3000	you, for Grafana
Backend	22	you
Backend	5432 6379 5672	nobody on the internet

5432, 6379, and 5672 are used only on the Docker network. Do not add them to the security group.

4. Clone

Step 1. Put the repository on the server

Clone the same repository on every server that will run a piece of RelayMart.

```
git clone https://github.com/YOUR_USER/relaymart.git
cd relaymart
ls docker-compose.yml docker-compose.frontend.yml scripts/up-backend.sh
```

Expected: All three paths exist. If docker-compose.yml is inside APP, cd APP and stay there. Every command below assumes that directory.

Do not copy node_modules from a laptop. The Dockerfiles install dependencies inside the image.

5. Values you set, and values you leave alone

Leave these alone for Compose

- Service ports, Docker network names, and the files under services/ and gateway/.
- db/init.sql. Compose loads it into Postgres on first start.
- Postgres user relay, database relaymart, and RabbitMQ user relay. They match the Compose file.

Set this when the browser is not on the server

GATEWAY_PUBLIC_URL is the address the shopper's browser uses to reach the gateway. It is not a Docker hostname.

```
GATEWAY_PUBLIC_URL=http://BACKEND_PUBLIC_IP:4000
```

Use http://127.0.0.1:4000 only when you open the shop on the server itself, or through an SSH tunnel to your laptop.

Set these only for ECS and EKS

- image_prefix: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart
- database_url: postgres://relay:PASSWORD@RDS_ENDPOINT:5432/relaymart
- redis_url: redis://ELASTICACHE_ENDPOINT:6379
- rabbit_url: leave empty until you have a broker, or set amqp://user:pass@host:5672
- vpc_id, subnet_ids, and the ECS task execution role ARN
- In deploy/ecs/elasticache.tf, replace the placeholder RDS password before the first apply.

- In deploy/eks/render.mjs, replace the Compose hostnames in DATABASE_URL, REDIS_URL, and RABBIT_URL, then render again.

6. Part A. One server

Step 2. Start the full stack

This builds 17 images and starts Postgres, Redis, RabbitMQ, Elasticsearch, Kibana, Prometheus, and Grafana.

```
docker compose up --build -d
docker compose ps
```

Expected: gateway, auth, postgres, redis, and rabbitmq become healthy. Elasticsearch can take two minutes. The first build is the slow part.

If auth or tracking restarts:

```
docker compose logs --tail 80 auth tracking postgres
```

Step 3. Prove the gateway

```
curl -s http://127.0.0.1:4000/health
```

Expected: {"ok":true,"service":"gateway"}

Step 4. Open the shop

From your laptop use http://PUBLIC_IP:4173. If the page loads but register fails, the shop is still calling 127.0.0.1. Recreate the frontend with the public URL:

```
GATEWAY_PUBLIC_URL=http://PUBLIC_IP:4000 docker compose up -d frontend
```

Step 5. Register, verify, log in, and track

Use a new email if you run register more than once.

```
curl -s -X POST http://127.0.0.1:4000/api/register \
  -H 'content-type: application/json' \
  -d '{"name":"Asha Menon","email":"asha@example.com","mobile":"9845012345","password"
:"Relaymart123"}'
```

Expected: HTTP 201 and a user id.

```
curl -s -X POST http://127.0.0.1:4000/api/verify/email/send \
  -H 'content-type: application/json' \
  -d '{"email":"asha@example.com"}'
docker compose logs --tail 30 verification
```

Expected: A JSON line msg otp_issued with a 6-digit code. It expires in 300 seconds. It is not sent as email.

```
curl -s -X POST http://127.0.0.1:4000/api/verify/email/confirm \
  -H 'content-type: application/json' \
  -d '{"email":"asha@example.com","code":"THE_CODE"}'
```

Repeat send and confirm for the mobile. Login is rejected until both succeed.

```
curl -s -X POST http://127.0.0.1:4000/api/verify/mobile/send \
  -H 'content-type: application/json' \
  -d '{"mobile":"9845012345"}'
docker compose logs --tail 20 verification
curl -s -X POST http://127.0.0.1:4000/api/verify/mobile/confirm \
  -H 'content-type: application/json' \
  -d '{"mobile":"9845012345","code":"THE_CODE"}'
curl -s -X POST http://127.0.0.1:4000/api/login \
  -H 'content-type: application/json' \
  -d '{"email":"asha@example.com","password":"Relaymart123"}'
```

Expected: A token. invalid_login means the password did not match. verification_required means an OTP is still missing.

```
curl -s -X POST http://127.0.0.1:4000/api/tracking/shipments \
  -H 'content-type: application/json' \
```

```
  -d '{"orderId":"ord_demo","pincode":"560001"}'
```

Copy the id, which starts with trk_, into the scan call.

```
curl -s -X POST http://127.0.0.1:4000/api/tracking/shipments/trk_ab12cd34/scan \
  -H 'content-type: application/json' \
  -d '{"status":"picked_up","note":"hub scan"}'
```

```
curl -s -X POST http://127.0.0.1:4000/api/checkout \
  -H 'content-type: application/json' \
  -d '{"lines":[{"productId":"meadow-tee","qty":1}],"method":"upi","pincode":"560001",
"coupon":"STUDIO"}'
```

Expected: HTTP 201, an order id, and a trace. STUDIO takes 100 off an order from 999.

Step 6. Read the logs in Kibana

Open http://PUBLIC_IP:5601. Create a data view named relaymart, index relaymart-*, time field @timestamp. In Discover, last 15 minutes:

```
service: "verification" and msg: "otp_issued"
```

Expected: The same code you read from docker compose logs. Also filter service auth and msg login, and service tracking and msg scan.

Grafana is on port 3000. The user is admin. The password is GF_SECURITY_ADMIN_PASSWORD in docker-compose.yml. RabbitMQ management is on port 15672, user relay, password relay. Prometheus is on port 9090. Query http_requests_total.

7. Part B. Frontend on one server, backend on another

Clone the repository on both machines. Do not change service code. The browser downloads the page from server 1 and then calls server 2 on port 4000. The frontend process does not proxy those API calls, so server 2 must accept port 4000 from the internet, not only from server 1.

Step 7. Start the backend

On the larger server. This starts the gateway, all 15 services, Postgres, Redis, RabbitMQ, and the log stack. It does not start the shop.

```
sh scripts/up-backend.sh
docker compose ps
curl -s http://127.0.0.1:4000/health
```

Expected: ok true, and no frontend container in docker compose ps.

Step 8. Start the frontend

On the second server. Replace the IP with the backend public IP.

```
GATEWAY_PUBLIC_URL=http://BACKEND_PUBLIC_IP:4000 sh scripts/up-frontend.sh
```

Expected: A frontend container listening on 4173.

Step 9. Prove the split

On your laptop open http://FRONTEND_PUBLIC_IP:4173 and register. If the button fails, the page is calling the wrong host. On the frontend server check the value that was baked into the page:

```
docker logs $(docker ps -q --filter name=frontend) | head
```

The running process does not print the URL on each request. Trust the environment you passed. Recreate the container if you change it:

```
GATEWAY_PUBLIC_URL=http://BACKEND_PUBLIC_IP:4000 \
  docker compose -f docker-compose.frontend.yml up -d --force-recreate
```

Then repeat the register, OTP, login, tracking, and checkout calls from Part A against http://BACKEND_PUBLIC_IP:4000. Read OTP codes on the backend server:

```
docker compose logs --tail 30 verification
```

8. Part C. Push images to ECR

Run this on a machine that already has Docker and the AWS CLI, after Part A is healthy. The script creates 17 repositories and pushes the tag latest.

Step 10. Confirm the AWS identity

```
aws sts get-caller-identity
```

Expected: Your account id and the user or role you expect.

Step 11. Build and push

Replace the account and region.

```
AWS_ACCOUNT_ID=123456789012 AWS_REGION=ap-south-1 sh scripts/ecr-push.sh
```

Step 12. Count the repositories

```
aws ecr describe-repositories --region ap-south-1 \
  --query 'repositories[?starts_with(repositoryName, `relaymart/`)].repositoryName' \
  --output text
```

Expected: Seventeen names, frontend through tracking.

The prefix for Terraform and for EKS is:

```
123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart
```

The IAM user needs ecr:GetAuthorizationToken, ecr:CreateRepository, ecr:DescribeRepositories, and ecr:PutImage, plus the layer upload actions. The ECS execution role needs permission to pull from these repositories and to write the log group /relaymart/mesh.

9. Part D. Database and cache on AWS

Compose created Postgres and Redis for you. ECS and EKS do not. On AWS, Postgres is RDS and Redis is ElastiCache. RabbitMQ is not created by the Terraform in this repository. Leave rabbit_url empty until you have Amazon MQ or your own broker. Auth and verification still run without it. Events are then not published.

Step 13. Set the RDS password

Open deploy/ecs/elasticache.tf and replace the placeholder password. Do this before apply.

Step 14. Create the data services and the cluster

From deploy/ecs. You need a VPC, two subnets, and an ECS task execution role. Auth will fail on this first apply because the database URL is still empty. That is expected.

```
cd deploy/ecs
terraform init
terraform apply \
  -var vpc_id=vpc-xxxx \
  -var 'subnet_ids=["subnet-a","subnet-b"]' \
  -var execution_role_arn=arn:aws:iam::123456789012:role/ecsTaskExecutionRole \
  -var image_prefix=123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart \
  -var create_rds=true \
  -var create_elasticache=true
```

```
terraform output rds_endpoint
terraform output elasticache_endpoint
```

Step 15. Load the schema

Compose did this. RDS does not.

```
psql "postgres://relay:YOUR_PASSWORD@RDS_ENDPOINT:5432/relaymart" \
  -f ../../db/init.sql
```

Expected: Tables users, payments, shipments, and tracking_events.

The security group in this Terraform allows 5432 and 6379 from 10.0.0.0/8. A database outside that range will not match, and auth will keep restarting.

10. Part E. ECS

Step 16. Apply with the real URLs

Leave elasticsearch_host empty the first time. Logs stay in CloudWatch. Set it later to a host the tasks can reach on port 9200.

```
terraform apply \
  -var vpc_id=vpc-xxxx \
  -var 'subnet_ids=["subnet-a","subnet-b"]' \
  -var execution_role_arn=arn:aws:iam::123456789012:role/ecsTaskExecutionRole \
  -var image_prefix=123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart \
  -var database_url='postgres://relay:YOUR_PASSWORD@RDS_ENDPOINT:5432/relaymart' \
  -var redis_url='redis://ELASTICACHE_ENDPOINT:6379' \
  -var rabbit_url='' \
  -var elasticsearch_host=''
```

Expected: Cluster relaymart. One Fargate service per image. Names resolve as gateway.relaymart.local and auth.relaymart.local inside the VPC. frontend and gateway receive a public IP. The others do not.

Step 17. Wait until the desired count is running

```
aws ecs describe-services --cluster relaymart --region ap-south-1 \
  --services auth gateway frontend \
  --query 'services[].{name:serviceName,running:runningCount,desired:desiredCount}'
```

Expected: running equals desired. gateway and frontend want 2. auth wants 1.

Step 18. Call the public gateway

Find the network interface, then its public IPv4 address in EC2.

```
TASK=$(aws ecs list-tasks --cluster relaymart --service-name gateway \
  --region ap-south-1 --query 'taskArns[0]' --output text)
aws ecs describe-tasks --cluster relaymart --tasks "$TASK" \
  --region ap-south-1 --query 'tasks[0].attachments[0].details'
```

```
curl -s http://GATEWAY_PUBLIC_IP:4000/health
```

Repeat the Part A register, OTP, login, tracking, and checkout calls against that IP. The frontend task also has a public IP on port 4173. Its GATEWAY_PUBLIC_URL inside the task is the Cloud Map name, which a laptop cannot resolve. For a public shop, point the frontend task environment at http://GATEWAY_PUBLIC_IP:4000 and open port 4173 in the security group to your users.

Step 19. Read ECS logs

```
aws logs tail /relaymart/mesh --region ap-south-1 --follow \
  --filter-pattern otp_issued
```

Expected: The same otp_issued JSON line.

When you set elasticsearch_host, Terraform adds a Fluent Bit sidecar on every task and ships logs to that host, index prefix relaymart. The sample uses TLS off. Use TLS for Amazon OpenSearch. CloudWatch still stores the sidecar's own logs.

11. Part F. EKS

The node role must be able to pull from ECR. AmazonEC2ContainerRegistryReadOnly on the node role is enough.

Step 20. Point the manifests at ECR

IMAGE_PREFIX has no trailing slash and no service name. Run this from the repository root, not from deploy/ecs.

```
IMAGE_PREFIX=123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart \
  node deploy/eks/render.mjs
kubectl apply -f deploy/eks/mesh.yaml
```

Expected: Namespace relaymart. One Deployment and one Service per image. The gateway has 2 replicas and an autoscaler. The Ingress host is relaymart.local.

Step 21. Point data URLs at RDS and ElastiCache

Edit DATABASE_URL, REDIS_URL, and RABBIT_URL in deploy/eks/render.mjs. The file ships with the Compose hostnames. Render again and apply again. Rollouts follow.

```
kubectl -n relaymart rollout status deploy/auth
kubectl -n relaymart rollout status deploy/gateway
kubectl -n relaymart get deploy
```

Step 22. Prove it

```
kubectl -n relaymart port-forward svc/gateway 4000:4000
```

In a second terminal, repeat the Part A curls against http://127.0.0.1:4000. A 201 from register means the pod reached RDS. Port-forward svc/frontend 4173:4173 for the shop. If you use the Ingress, map relaymart.local to the load balancer and keep GATEWAY_PUBLIC_URL equal to the URL the browser will call.

12. Part G. Logs on EKS

Step 23. Install Elasticsearch, Kibana, and Fluent Bit

Elasticsearch requests 1 GB. A node smaller than that leaves the pod Pending.

```
kubectl apply -f deploy/eks/efk.yaml
kubectl -n logging get pods
```

Expected: elasticsearch and kibana Running, and one fluent-bit pod on each node.

Step 24. Open Kibana and confirm a live line

```
kubectl -n logging port-forward svc/kibana 5601:5601
```

Create the data view relaymart-* again. Send another email OTP through the gateway port-forward. In Discover:

```
service: "verification" and msg: "otp_issued"
```

Expected: A new line within a few seconds. That is the deploy: ECR images on EKS, and the log in Kibana.

```
kubectl -n logging logs daemonset/fluent-bit --tail 30
```

Prometheus is separate. The Deployments are annotated for a scrape on /metrics. Install a Prometheus stack in the cluster if you want the same Grafana boards you used on port 3000.

13. Done when all of these pass

Check	Pass
docker compose ps	auth, gateway, postgres, redis healthy
GET /health on :4000	ok true
POST /api/register	201 and a user id
verification log	otp_issued, then confirm returns verified
POST /api/login	a token after both codes
POST /api/checkout	201 and a trace
ECR	17 relaymart repositories
ECS services	running count equals desired
kubectl get deploy -n relaymart	ready
Kibana relaymart-*	otp_issued from that environment

14. When it fails

auth restarts

DATABASE_URL is wrong, the security group blocks 5432, or init.sql was not loaded. Read docker compose logs auth or the ECS stopped-task reason.

The shop page loads and every button fails

GATEWAY_PUBLIC_URL is 127.0.0.1 or a Docker name. Set it to the public gateway URL and recreate the frontend.

Login returns verification_required

Confirm email and mobile. Registering the same email again returns email_or_mobile_taken.

Kibana is empty but docker compose logs shows JSON

Fluent Bit on Compose reads /var/lib/docker/containers. Another container runtime will not match that path. On EKS, the fluent-bit DaemonSet must be Running and must see /var/log/containers.

ECS CannotPullContainerError

image_prefix does not match the account that owns the repositories, or the execution role cannot pull from ECR.

EKS image pull fails

The node role cannot read ECR, or render.mjs was run without IMAGE_PREFIX, so the manifest still says relaymart/frontend:latest.

elasticsearch Pending

The node does not have 1 GB free. Use a larger node, or lower the memory request in deploy/eks/efk.yaml.

15. Stop

```
docker compose down -v
```

On the frontend server:

```
docker compose -f docker-compose.frontend.yml down
```

ECS, RDS, and ElastiCache. This deletes the RDS database. The file sets skip_final_snapshot, so there is no snapshot.

```
cd deploy/ecs
terraform destroy
```

```
kubectl delete -f deploy/eks/mesh.yaml
kubectl delete -f deploy/eks/efk.yaml
```

16. Order of work

- Install Docker and clone.
- Part A on one server until register, both codes, login, tracking, and checkout pass.
- Part B only if the shop and the API must be on different machines.
- Part C after Part A. Push the images that you already proved.
- Part D, then Part E. Load init.sql before you expect auth to stay up.
- Part F and Part G when the target is Kubernetes. Kibana is the last check, not the first.